

University of Pittsburgh

Introduction to Operating Systems
CS/COE 1550

Fall 2021

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines:
  - 10/25: Project 2 (extended deadline)
  - 10/29: Homework 6
  - 11/8: Lab 3

Last Lecture …

- Deadlock prevention

Muddiest Points (Banker's Algorithm)

- what are the three steps of the bankers algorithm?
- Could you go over the Bankers algorithm example again?
- For the safe/unsafe algorithm, does that mean we need to know the resources that are available, keep track of the used ones, and also know how many each process will have in the future?
- It seems like this detection algorithm has a lot of overhead. Are there any ways of reducing the time spent doing these checks
- can we run the banker's algorithm or some alternative as a static analysis tool? like build this kind of check into the compiler or something instead of running it for each request during runtime
- If a state is determined unsafe, does that always mean that the resource will not be allocated or is it possible to still allocate it and take the risk
- What does the E, P and A stand for in the bankers algorithm
- if bankers algorithm finds a valid path and grants the resource, can a deadlock still occur if a different path is followed than the one bankers algorithm found?
- I'm a little confused on what the want and hold tables represent
- Why is the runtime n^2 * n for the wait algorithm
- What is the difference between wanted and max again?
- Tracing bankers algorithms with multiple resource instances

Muddiest Points (Resource Trajectory)

- the l1, l2 l3 … in the resource trajectory graph
- resource trajectory graphs with more than 2 resources
- How to prevent a resource trajectory that would cause a deadlock

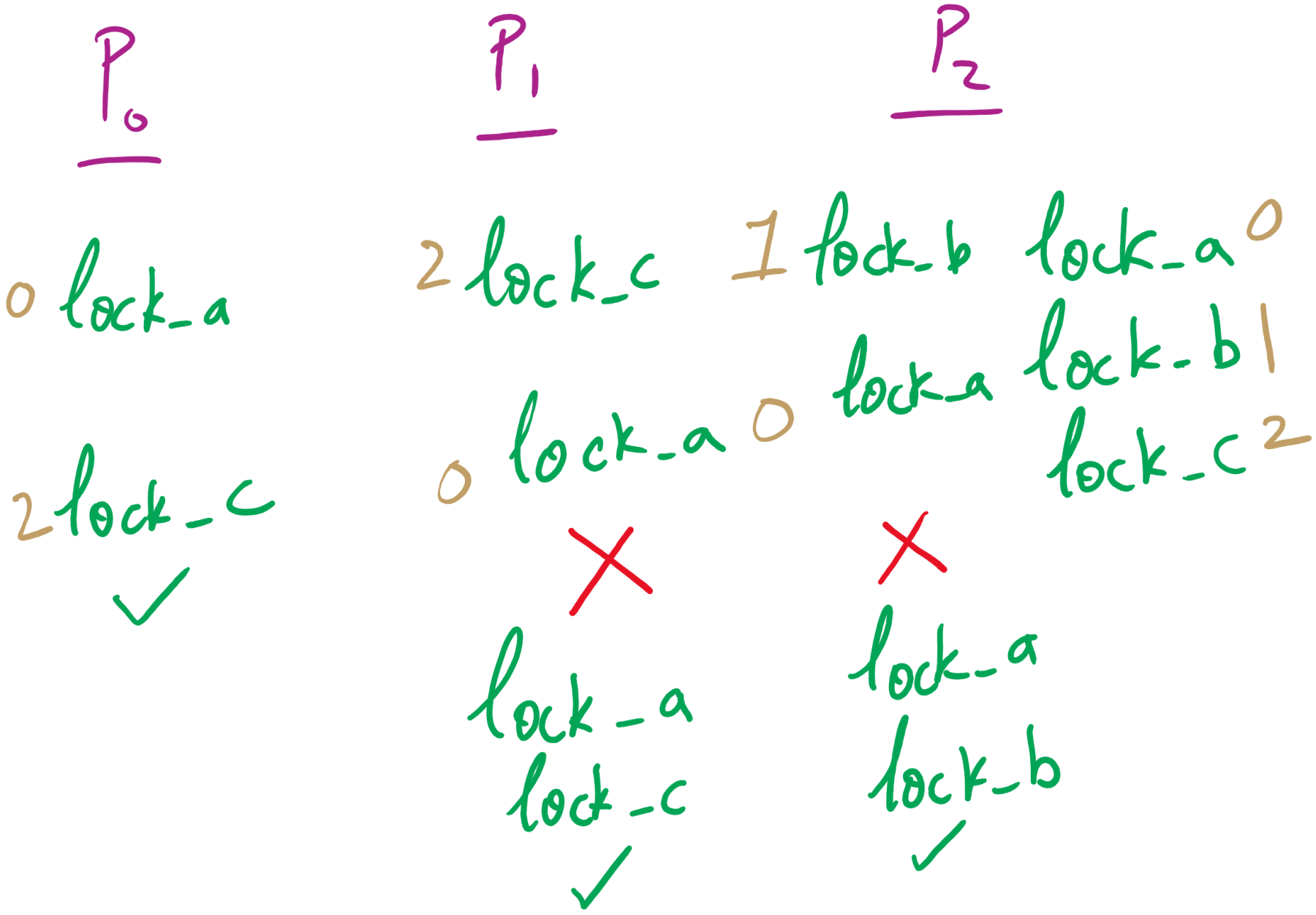
Muddiest Points (General Deadlock)

- breaking circular wait seems almost always better than breaking hold and wait since breaking hold and wait requires you take all resources you need whereas breaking circular wait only makes you take some extra resources before you need them
- Starvation vs deadlock
- Which detection alg is fastest?
- Where do deadlock detection algorithms exist? (Operating system or external application.) When do they run? Runtime or compile time? Also, how do deadlock avoidance algorithms interfere to make sure a program doesn't deadlock? Are they integrated fully into the application?
- how do we know what resources are available to use and the best way to choose who gets what resources
- What is better practice, deadlock avoidance or deadlock detection?

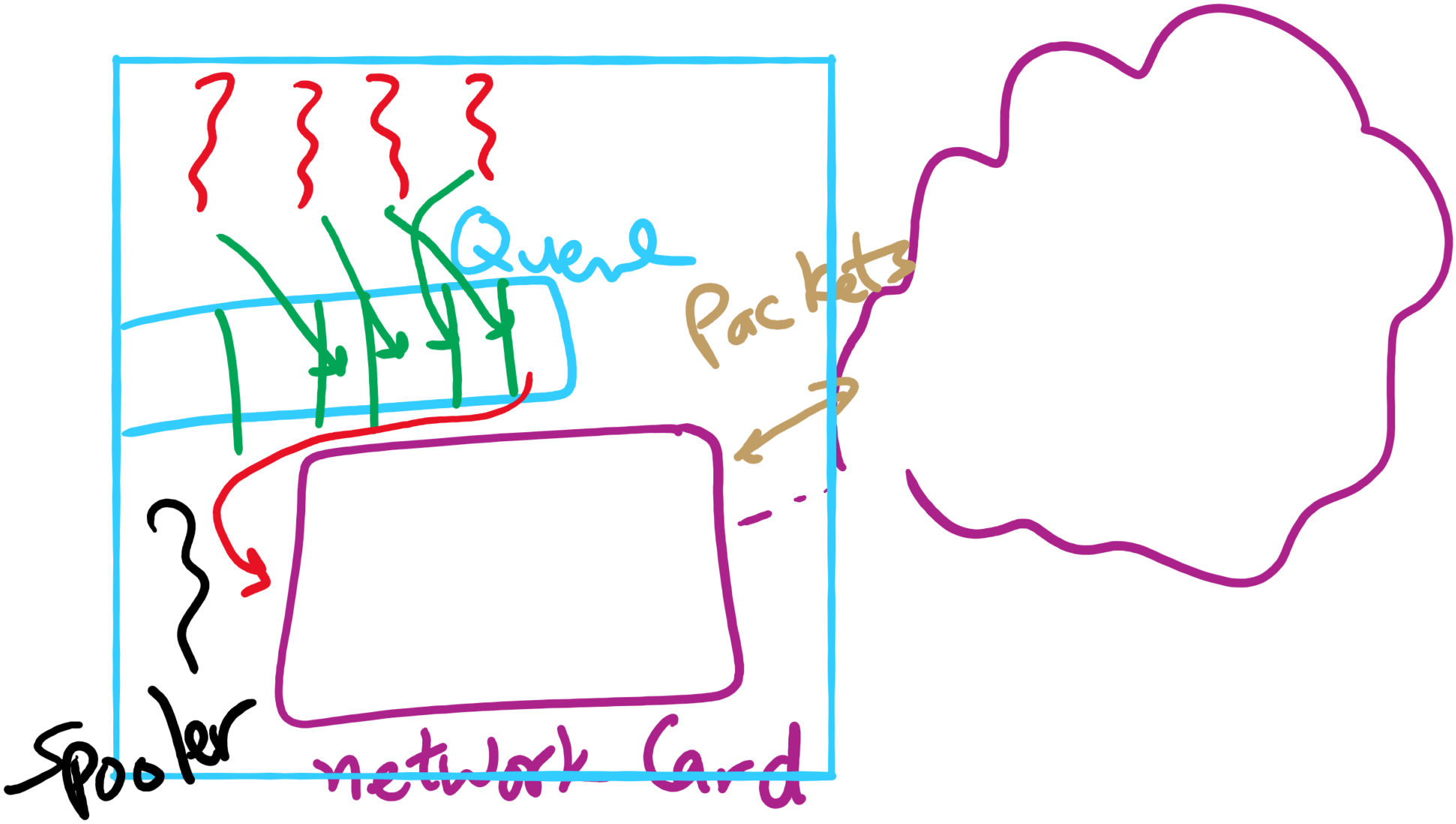
Muddiest Points (Deadlock Prevention)

- When checking a program to see it is free of deadlocks, you have to look for whether one of the four condtions is being broken somewhere?
- Why do you only need to attack one of the 4 conditions to prevent deadlock?
- How to know which deadlock prevention technique to use.
- How spooling works
  - What exactly is spooling?
  - Spooling in applications
  - When is it appropriate to spool? How do you know?
- How non-resource deadlocks occur
- In regard to attacking "no preemption", how would you determine whether it's viable to take a resource away from a process?
- how 2 phase locking avoids the hold and wait condition?

Attacking Circular Wait



Spooling Example



Today …

- CPU Scheduling